

DiSwitch: Global-to-Local Policy Distillation with Risk-Switched Predictive Recovery for Decentralized UAV Routing

Kyung Eun Kim ¹ and Howon Lee ^{1,*}

¹ Department of Military Digital Convergence, Ajou University, Suwon 16499, Republic of Korea

* Correspondence: [institutional e-mail to be provided](#)

Abstract

Flying ad hoc networks require reliable next-hop selection under changing connectivity and limited local information. We present DiSwitch (Distilled Policy Switch), which transfers a graph-based reinforcement-learning teacher's preferences to a local student and combines a geographic-residual policy with a predictive prior through a risk switch. The teacher is used offline; online decisions consume relay and one-hop features under the stated observation contract. In a common simulator, eight implementations were evaluated using five training seeds, 14 scenarios, and 200 episodes per seed–scenario cell. DiSwitch achieved a scenario-macro connected-pair packet delivery ratio of 0.9053 and a deadline-delivery ratio of 0.8376. Relative to adapted Evo-QGeo, the paired differences were 1.84 and 2.23 percentage points, respectively. Successful-packet delay is reported in simulator steps and separately from software decision latency. In one Google Colab A100 session, mean seed-specific batch-one p95 decision latency was 5.836 ms on CUDA and 2.208 ms on the host CPU. A compact distilled variant reduced CUDA latency to 2.005 ms but did not satisfy the study's 0.5-percentage-point reliability tolerance. The findings support the evaluated local policy composition and expose a reliability–latency trade-off. They do not establish performance on airborne hardware or superiority over independently reproduced wireless protocol implementations.

Keywords: UAV routing; policy distillation; reinforcement learning; FANET; decision latency

Editorial review copy. Red text and red-framed figures are candidates for deletion or relocation to the supplement. Blue text identifies information requiring author verification. Black text is revised manuscript content.

1. Introduction

Unmanned aerial vehicles can form infrastructure-free multihop networks for disaster response, sensing, inspection, and temporary coverage. The routing graph in such a flying ad hoc network (FANET) evolves quickly because relative motion changes both connectivity and link quality. A next hop that offers immediate geographic progress may disappear before transmission or may lead to a relay with no stable onward neighbor. Reliable routing therefore needs enough structural and temporal context to anticipate failure, yet an onboard relay cannot assume continuous access to a global graph, a centralized controller, or unlimited compute.

Reinforcement learning (RL) has been applied to joint trajectory and routing control, age-of-information optimization, trusted routing, topology-aware recovery, graph-based

Received:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Drones* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\) license](#).

forwarding, and high-speed FANETs [1–6]. These studies demonstrate the value of learned adaptation but leave two related deployment questions. First, can global training information be converted into a policy that uses only the observation available at the forwarding relay? Second, if two local experts are needed to cover ordinary and imminent-link-failure conditions, how should their runtime be engineered and measured without confusing network end-to-end delay with neural decision time?

This work addresses both questions. The proposed method, DiSwitch, is learned through a sequence of global-teacher, local-distillation, geographic-prior, predictive-feature, and risk-calibration stages. Its deployed policy contains a normal geographic-residual branch and a predictive-prior branch. The normal branch retains efficient routing in common states; the predictive branch recovers states in which the nominal link is locally unsafe. A calibrated switch uses only local features and the masked branch preferences. No global graph or teacher messages are required during execution.

The deployed implementation evaluates each branch once and reuses its outputs for action selection and diagnostics. This implementation detail supports reproducible timing. The routing contribution is the combination of local distilled preferences, predictive risk features, and calibrated switching; a comparison with an earlier instrumented wrapper is reported only as an internal implementation check.

The study is guided by four research questions:

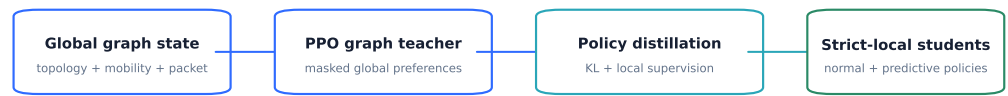
1. Does strict-local global-to-local distillation with risk-switched recovery improve delivery reliability over conventional, value-based, and graph-RL references in a common simulator?
2. Which component of the proposed method accounts for the observed improvement, and where does the method fail?
3. How much latency is removed by semantics-preserving computation reuse, and why can a large GPU be slower than its host CPU for batch-one routing?
4. Can a faster distilled or conditionally evaluated policy replace DiSwitch without violating a study-defined delivery-quality gate?

The principal contributions are as follows.

1. We formulate a strict deployment contract that allows privileged graph information for offline learning but restricts online forwarding to relay-local and one-hop information.
2. We combine a geographic-residual normal policy with a predictive local policy through an interpretable risk switch based on margin, link lifetime, and onward lifetime.
3. We specify a single-evaluation implementation of the two-branch decision rule and measure its batch-one runtime. Removing repeated diagnostic calls is an implementation optimization, not a separate routing algorithm contribution.
4. We evaluate eight methods over five seeds, 14 scenarios, and 200 episodes per seed–scenario cell, using seed-paired scenario-macro inference and explicit definitions for reliability, conditional delay, an energy proxy, and policy-input footprint.
5. We benchmark CPU and A100 CUDA batch-one execution in the same session and profile device-to-host scalar synchronization. We also report negative results: calibrated early exit is behaviorally accurate on the evaluated trajectories but slower, whereas Fast-DiSwitch is faster but fails the reliability gate.

Figures 1 and 2 separate the privileged training path from the decentralized forwarding path so that each data flow remains readable at journal-column scale.

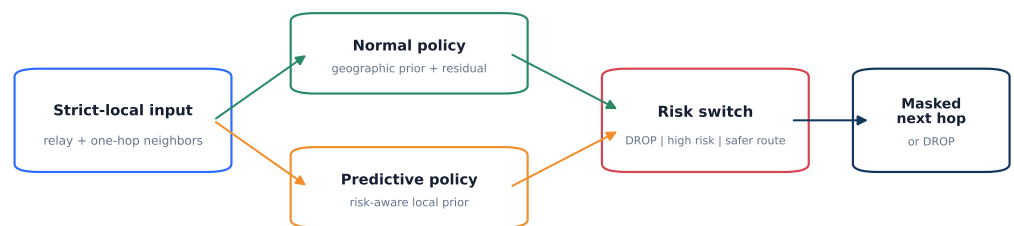
Privileged training: global knowledge is distilled into local policies



The teacher and global graph are removed after training; deployment receives no controller messages.

Figure 1. Privileged training stage of DiSwitch. Global graph knowledge is transferred through policy distillation, while the teacher is removed before deployment.

Decentralized batch-1 forwarding: local evidence branches and reconverges at the switch



All switch inputs are computed from strict-local observations and the two policy outputs.

Figure 2. Strict-local deployment stage of DiSwitch. Local input splits into normal and predictive policies, whose outputs reconverge at the risk switch before masked next-hop selection.

2. Related Work and Research Gap

2.1. Learning-Based UAV and FANET Routing

Topology-aware Q-learning, Q-learning with rate control, and improved multihop Q-learning provide lightweight value-based adaptation [4,7,8]. Predictive-Q explicitly couples mobility and interference with dual-path preparation [9]; its reported lightweight decision path motivates our predictive recovery design, but a destination-indexed table and a learned dual-branch policy have different state scaling and runtime behavior. Recent deep RL methods use recurrent DQN, graph attention, multi-agent actors, and broader joint optimization [5,6,10,11]. The UAV literature therefore establishes that topology and motion prediction matter, but it does not by itself guarantee decentralized observability or low batch-one inference latency.

Some methods optimize substantially broader actions. Joint trajectory, frequency allocation, routing, scheduling, power, or buffering can improve a system objective [1,2,11,12]. These settings are complementary, not numerically interchangeable, with per-hop next-hop selection. Similarly, resilient cluster maintenance and routing in harsh environments provides strong venue-level evidence that graph RL is relevant to UAV resilience [13], but its cluster-level maintenance objective differs from our strict-local forwarding contract.

2.2. Deployment and Latency Reporting

Only a minority of the 45 records in the project literature database reports a scoped routing-decision runtime. Many report network end-to-end delay, which includes queueing, medium access, retransmission, propagation, and multihop forwarding. RLFR includes a Raspberry Pi 4B testbed and local forwarding design [14]; RoutePPO reports an eBPF/P4 forwarding-plane update path [15]; an ultra-high-speed recurrent method reports online decision time [6]; and Predictive-Q reports both per-packet and per-slot computational quantities [9]. Hardware, batch size, synchronization, and timed scope are not uniform.

Consequently, those values are contextual references rather than fair direct baselines for the present PyTorch policy.

The literature counts summarize a project convenience sample. The database count is a project audit, not a systematic-review meta-analysis: tags are inherited from the curated records, and the decision-latency count was manually restricted to explicitly scoped compute or forwarding decisions.

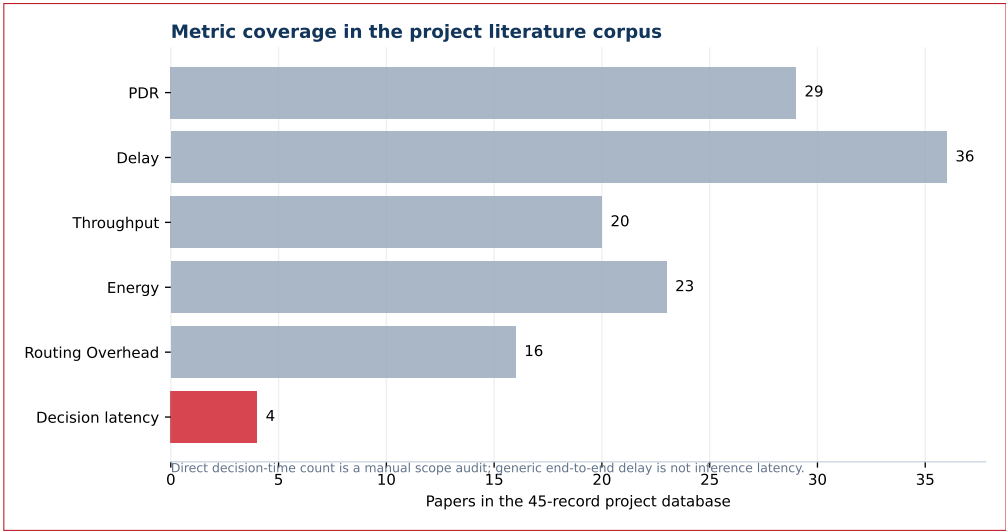


Figure 3. Deletion/move candidate D02. Convenience-corpus counts; omit or move to a literature-audit appendix. Metric coverage in the 45-paper project corpus. Generic network delay is common, while directly scoped decision latency is uncommon.

2.3. Positioning and Novelty Boundary

Table 1 states the novelty boundary conservatively. The work does not claim the first use of RL, graph encoders, policy distillation, prediction, or conditional computation in isolation. Its contribution is their deployment-constrained composition and the semantics-preserving-versus-approximate validation discipline.

Table 1. Claim-gap matrix used to delimit the contribution.

Literature capability	Remaining gap	Proposed element	Evidence in this paper
Graph or multi-agent RL	Global or exchanged state may be required online	Privileged teacher, strict-local student	Observation contract and ablation Scenario heatmap and component effects
Predictive or dual-path routing	Prediction can add cost or override a sound nominal action	Calibrated risk-switched predictive recovery	DiSwitch-Fast-DiSwitch paired five-seed test Same-session A100 benchmark Legacy-DiSwitch equivalence and latency
Compact learned routing	Faster approximation can alter route outcomes	Predeclared reliability gate	
GPU inference	Large accelerators are often assumed to be faster	Synchronized batch-one CPU/CUDA scope	
Instrumented policy adapters	Diagnostic work can duplicate inference invisibly	Semantics-preserving tensor reuse	

3. System Model and Problem Formulation

3.1. Dynamic FANET and Local Action Space

At decision step t , the network is a dynamic graph $\mathcal{G}_t = (\mathcal{V}, \mathcal{E}_t)$. A directed edge $(u, v) \in \mathcal{E}_t$ indicates that relay u can currently attempt a transmission to v . For a packet with destination d , the action space at relay u is

$$\mathcal{A}_u(t) = \mathcal{N}_u(t) \cup \{\text{DROP}\}, \quad (1)$$

where $\mathcal{N}_u(t)$ is the set of reachable one-hop neighbors. Padding permits a fixed maximum number of candidates, and a Boolean mask $M_u(t)$ excludes padding, disconnected candidates, and prohibited actions. The explicit DROP action prevents an invalid padded index from being interpreted as a relay.

The teacher observes a privileged state s_t that may contain the full graph, positions or motion states, link features, the current relay, destination, and packet context. The deployed student receives

$$o_u(t) = (x_u, x_d, x_p, \{x_v, x_{uv}, f_v, r_v\}_{v \in \mathcal{N}_u(t)}, M_u), \quad (2)$$

where x_u , x_d , and x_p are relay, destination, and packet features; x_v and x_{uv} are neighbor and link features; f_v denotes bounded forwardability features; and $r_v = [m_v, \ell_v, q_v, o_v]$ contains normalized link margin, current-link lifetime, queue headroom, and best-onward-link lifetime. The deployment contract excludes the full adjacency matrix, remote hidden states, and teacher queries. However, simulator-local features do not by themselves establish that those features are locally measurable on a UAV. Destination state, neighbor queue information, and onward-connectivity summaries require an acquisition and freshness model; this study does not measure the radio cost of obtaining them.

3.2. Objective and Constraints

Let $Y_e \in \{0, 1\}$ denote delivery in episode e , $C_e \in \{0, 1\}$ indicate that the source–destination pair was initially connected, and D_e denote delivery before a deadline. We seek a local policy that maximizes reliability and deadline delivery while controlling delay and execution cost: The design prioritizes connected-pair and deadline delivery and reports execution time and conditional successful delay as separate outcomes. We do not claim to solve a calibrated scalar optimization problem that combines these quantities in different units. Approximate speed variants are judged by a separate acceptance rule. If Δ_C and Δ_D are seed-paired changes relative to DiSwitch, a candidate is eligible only if

$$\text{LB}(\Delta_C) \geq -0.005, \quad \text{LB}(\Delta_D) \geq -0.005, \quad (3)$$

where the bound is the lower endpoint of the two-sided 95% t interval used in the archived analysis. The tolerance is 0.005 in ratio units (0.5 percentage points). Delay and energy provide supporting diagnostics; no numeric acceptance margins were specified for them. We do not claim prospective preregistration of this rule. Failure to establish non-inferiority is not, by itself, a proof that degradation exceeds the tolerance.

4. Proposed Method

4.1. Privileged Global PPO Teacher

The graph-aware teacher is trained offline using proximal policy optimization (PPO) [16]. Given advantage estimate $\hat{A}_t$, probability ratio $\rho_t(\theta)$, and clip parameter ϵ , its actor loss is

$$\mathcal{L}_{\text{PPO}}(\theta) = -\mathbb{E}_t[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]. \quad (4)$$

The same local validity mask used by the student is applied to the teacher's candidate logits before probabilities are produced. The teacher is a source of preferences, not an optimal-route oracle and not a deployment dependency.

4.2. Masked Global-to-Local Distillation

For temperature τ , the teacher and student masked distributions are

$$\begin{aligned} p_T^\tau(a) &= \text{softmax}\left(\frac{z_T(a) + \log M(a)}{\tau}\right), \\ p_S^\tau(a) &= \text{softmax}\left(\frac{z_S(a) + \log M(a)}{\tau}\right). \end{aligned} \quad (5)$$

Here $M(a) \in \{0, 1\}$, $\log 0 = -\infty$, and normalization is over valid actions, with $\tau > 0$. The KL divergence is evaluated on that common support; DROP must remain a defined fallback when no neighbor is valid. The principal distillation term is [17]

$$\mathcal{L}_{\text{KD}} = \tau^2 D_{\text{KL}}(p_T^\tau \| p_S^\tau). \quad (6)$$

The auxiliary supervision sources considered in the project include teacher actions, shortest-path actions, and local risk-aware actions. [The release must identify the active loss weights, distillation temperature, reward terms, training budgets, and checkpoint-selection rule for every reported checkpoint.](#) A split by scenario and episode is required to prevent trajectory leakage; independence of tuning and evaluation must be documented with split manifests before a prospective validation claim is made.

4.3. Normal Geographic-Residual Branch

For candidate v , let $\Delta d_v = d(u, d) - d(v, d)$ be geographic progress. A shared local encoder $h_\theta(o_u, v)$ and pooled neighbor context $\bar{h}$ generate a residual score $r_\theta(o_u, v)$. With a learnable positive prior scale α and forwardability feature f_v , the normal logit is

$$z_v^N = \alpha \Delta d_v + w_f^\top f_v + r_\theta(o_u, v). \quad (7)$$

The residual corrects greedy behavior while the prior provides a stable inductive bias. The branch also emits an explicit drop logit, and the structural mask is applied after all scores are assembled.

4.4. Predictive Local Branch

The predictive branch uses the risk vector $r_v = [m_v, \ell_v, q_v, o_v]$. Its implementation is calibrated as a predictive-prior-only branch: the learned residual contribution is set to zero after checkpoint loading, preventing a learned residual from overriding a strong imminent-link-break signal. For calibrated gates $\gamma_m = 0.04$, $\gamma_\ell = 0.20$, and $\gamma_o = 0.20$, candidate danger is

$$D(v) = [\gamma_m - m_v]_+ + [\gamma_\ell - \ell_v]_+ + [\gamma_o - o_v]_+. \quad (8)$$

Queue headroom is available to the predictive prior but is intentionally not part of the switch danger in the verified implementation. This distinction matters for source-to-code consistency.

4.5. Risk Switch

Let K be the padded candidate capacity, with candidate slots $0, \dots, K-1$ and DROP index K . Let $a_N = \arg \max_a z_a^N$ and $a_P = \arg \max_a z_a^P$ after masking. For consistency with

the evaluated code, define $\iota(a) = \min(a, K - 1)$ and interpret the risk features indexed by a below as those at slot $\iota(a)$. This is an indexing convention for DROP, not a physical link-risk estimate. Define the safety gain

$$G(a_P, a_N) = (m_{a_P} + \ell_{a_P} + o_{a_P}) - (m_{a_N} + \ell_{a_N} + o_{a_N}). \quad (9)$$

For switch threshold $\eta = 0.05$, the predictive branch is selected when at least one neighbor exists and

$$S = \mathbf{1} \left[\begin{array}{l} a_N = \text{DROP} \vee D(a_N) > \eta \\ \vee (a_P \neq a_N \wedge G(a_P, a_N) > 0.10) \\ \wedge D(a_P) < D(a_N) \end{array} \right]. \quad (10)$$

When no valid neighbor exists, $S = 0$; if risk features are absent, the evaluated switch also defaults to the normal branch. The indexing convention can affect a predictive DROP choice and must be covered by regression tests before any change. The switch is a heuristic selection rule, not a guarantee of link safety. The final logits are $z = (1 - S)z^N + Sz^P$, followed by the unchanged action mask and softmax. Thus the switch protects the ordinary branch from unnecessary replacement but allows predictive recovery for a drop, a risky nominal link, or a demonstrably safer alternative.

4.6. DiSwitch Execution Reuse

The implementation uses the same logits for action selection and diagnostic extraction. The legacy instrumented path first obtained the action and then invoked diagnostics that recomputed both branch policies. DiSwitch returns a structured decision containing output logits, switch state, normal and predictive actions, normal probabilities, and diagnostics. Every diagnostic is computed from already available tensors.

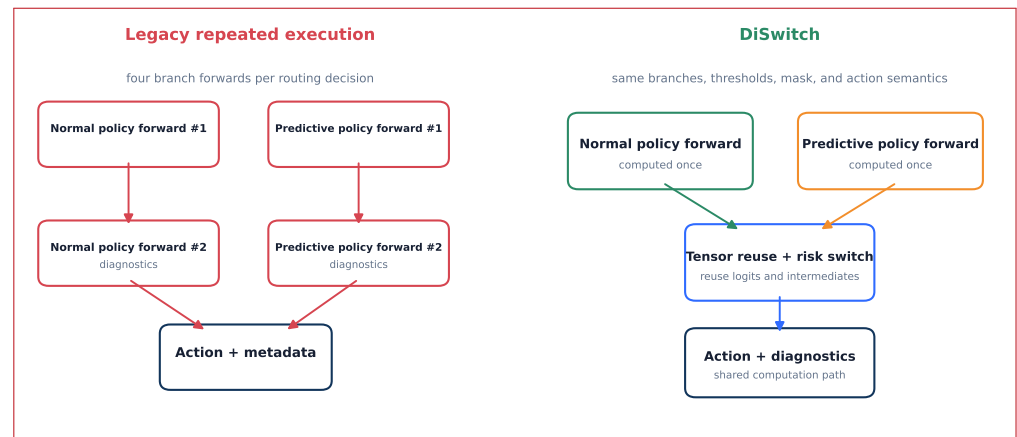


Figure 4. Deletion/move candidate D01. Internal wrapper comparison; move to implementation appendix. Legacy repeated execution versus DiSwitch. Tensor reuse removes duplicate branch evaluations without pruning a branch or approximating its logits.

Semantic-equivalence proposition. Assume deterministic evaluation mode, identical observations and parameters, and a legacy wrapper whose repeated diagnostic calls do not mutate policy state. Then DiSwitch returns the same masked logits, action, switch indicator, and deterministic diagnostics as the mathematical two-branch policy.

Justification. The normal and predictive modules are pure deterministic mappings in evaluation mode. Re-evaluating either mapping produces the same logits. Replacing the second evaluations with references to the first logits therefore leaves Equations (8)–(10), the final mask, and all diagnostics unchanged. The implementation is additionally protected by regression tests that count branch forwards and compare replayed outcomes. This is

an implementation equivalence argument under deterministic evaluation. Its associated wrapper-comparison diagram is marked for relocation to supplementary material.

Algorithm 1 gives the deployed decision path.

Algorithm 1 DiSwitch decision

Require: local observation o , action mask M , normal policy π_N , predictive policy π_P

- 1: $z^N, p^N \leftarrow \pi_N(o)$ ▷ one normal forward
- 2: $z^P, p^P \leftarrow \pi_P(o)$ ▷ one predictive forward
- 3: $a_N \leftarrow \arg \max \text{mask}(z^N, M); a_P \leftarrow \arg \max \text{mask}(z^P, M)$
- 4: compute $D(a_N)$, $D(a_P)$, and $G(a_P, a_N)$ using the slot map ι
- 5: $S \leftarrow \text{Equation (10)}$
- 6: $z \leftarrow \text{where}(S, z^P, z^N)$
- 7: $\tilde{z} \leftarrow \text{mask}(z, M); p \leftarrow \text{softmax}(\tilde{z})$
- 8: derive diagnostics from z^N, z^P, a_N, a_P, S without another forward
- 9: **return** $\arg \max \tilde{z}, p, S$, diagnostics

4.7. Runtime Model and Candidate Optimizations

Decision time is decomposed as

$$T_{\text{dec}} = T_{\text{pre}} + T_N + T_P + T_{\text{gate}} + T_{\text{mask}} + T_{\text{extract}} + T_{\text{sync}} + T_{\text{runtime}}. \quad (11)$$

Component timers diagnose bottlenecks but are not added to reconstruct end-to-end time because their instrumentation and synchronization boundaries differ. The primary statistic times the complete observation-to-action call.

Three alternatives were tested. Buffered DiSwitch reuses input-tensor storage. Calibrated Early Exit evaluates only the normal branch when its chosen candidate is not drop, a live candidate exists, and $D(a_N) = 0$. Its expected time is

$$\mathbb{E}[T_{EE}] = T_N + (1 - p_{\text{skip}})T_P + T_g, \quad (12)$$

so it helps only if $T_g < p_{\text{skip}}T_P$. Fast-DiSwitch distills DiSwitch into a compact single-pass network. It changes the policy family and must therefore pass Equation (3); it cannot inherit DiSwitch's behavioral evidence merely because it is trained from DiSwitch.

5. Experimental Methodology

5.1. Common-Simulator Routing Evaluation

The full comparison uses eight policies: AODV, OLSR, Greedy Geographic, adapted Evo-QGeo, adapted RDQN-HERP, GAT-GRU-DDQN, Fast-DiSwitch, and DiSwitch. "Adapted" means that the published organizing idea was implemented inside the common simulator; the values are not copied from the original papers and should not be read as a reproduction of their reported testbeds. All methods receive the simulator information permitted by their contracts and use the same episode and scenario schedule.

Five training seeds (42, 77, 123, 314, and 2718), 14 held-out or stress scenarios, and 200 episodes per seed–scenario cell produce 14,000 episodes per method and 112,000 episode rows in the combined archive. Scenarios cover nominal held-out mobility, stochastic link loss, high or extreme mobility, sparse connectivity, node-count shifts, structural geographic holes, predictive breaks, and predictive breaks with loss. Figure 5 summarizes the design.



Figure 5. Evaluation protocol. The seed, not the individual packet decision, is the primary inferential unit.

5.2. Metrics and Scope

For episode set $\mathcal{E}$, connected-pair PDR is

$$\text{PDR}_C = \frac{\sum_{e \in \mathcal{E}} Y_e C_e}{\sum_{e \in \mathcal{E}} C_e}, \quad (13)$$

and overall PDR is $|\mathcal{E}|^{-1} \sum_e Y_e$. Deadline delivery ratio is $|\mathcal{E}|^{-1} \sum_e D_e$. The p95 successful delay is the 0.95 quantile of forwarding steps among delivered packets only. A missing-delivery cell is reported as N/A, never as zero; otherwise a policy that drops every packet could appear to have zero delay. The step count is a simulator route-duration statistic. No conversion to physical end-to-end delay in milliseconds is supported without a calibrated link, MAC, queueing, and time-step model. The reported aggregate p95 is a mean of cell-level quantiles, not a quantile of the pooled packet population.

Energy per delivered packet is a simulator proxy that aggregates modeled transmission cost and a failure penalty under the synthesis aggregation. It is not a physical Joule measurement. Mean policy-input bytes is the in-memory tensor footprint consumed by the policy and is not wireless control-plane overhead. These qualifications are repeated in the tables because removing the scope would create invalid cross-paper claims.

5.3. Statistical Analysis

Episode outcomes are first aggregated within each seed and scenario, then averaged across the 14 scenarios. The primary reported mean is therefore a five-seed scenario-macro estimate. For method A and reference B , a directional paired difference is $d_s = x_{A,s} - x_{B,s}$ for higher-is-better metrics and $d_s = x_{B,s} - x_{A,s}$ for lower-is-better metrics. A two-sided 95% Student- t interval is formed across the five paired seed differences. Deterministic bootstrap intervals and exact sign-flip checks are retained in the latency analysis. With five pairs, the smallest attainable two-sided exact sign-flip p -value is $2/2^5 = 0.0625$; hence effect sizes and intervals, rather than a binary significance threshold, lead the interpretation.

The intervals are unadjusted for multiple comparisons. The strongest comparator was identified within the evaluated set, so these contrasts are descriptive rather than a prospectively specified family-wise superiority test. Zero between-training-seed variance for deterministic policies does not imply zero deployment uncertainty. A negative lower t bound for a nonnegative energy proxy reflects an unconstrained small-sample interval, not negative energy; a future replication should report a support-respecting interval.

The repository also contains a pooled full-analysis table whose weighting differs from the scenario-macro synthesis, especially for the failure-penalized energy metric. This paper uses the combined synthesis consistently. The alternate aggregation is disclosed in the statistical audit rather than mixed selectively into figures.

5.4. Decision-Latency Benchmark and Profiling

The runtime measurement environment is a Google Colab session with an NVIDIA A100-SXM4-40GB, PyTorch 2.11.0+cu128, CUDA runtime 12.8, and Python 3.13.15. CPU and CUDA variants were executed in the same session, with batch size one, one CPU thread, 50 warm-up calls, 2,000 timed repetitions per seed and component, and five checkpoints. CUDA measurements synchronize around the timed end-to-end call. The primary latency statistic is the mean across seed-specific p95 values. Cold start is stored separately.

Operator profiling is a distinct short run on seed 42 and the held-out-medium scenario after 50 warm-up calls, with 30 profiled decisions. Profiler event totals attribute bottlenecks but are not substituted for the 2,000-repeat latency statistic. The study records code bundle, checkpoint archive, result archive, hardware, package freeze, and SHA-256 hashes.

Deletion/move candidate D05. An attempted expanded rerun for additional p90, maximum, coefficient-of-variation, and peak-device-memory fields was interrupted when the backend reclaimed the session; no values are imputed from that failed job.

6. Results

6.1. Overall Routing Performance

Table 2 and Figure 6 show the scenario-macro results. DiSwitch has the highest connected-pair PDR and deadline delivery among the eight common-simulator methods. It does not universally minimize every cost: AODV and OLSR have smaller policy inputs and lower conditional successful delay, with the latter conditional on each method’s own delivered-packet subset. These simulator-step differences cannot be attributed to neural inference time, which was benchmarked separately. Reliability and delay must therefore be interpreted jointly.

Table 2. Five-seed, 14-scenario macro means. Parentheses show 95% CI where non-degenerate. Energy is a simulator proxy; p95 delay is conditional on successful delivery. Bold identifies the proposed method, not the winner of each column.

Method	Connected PDR	Deadline ratio	p95 delay (steps)	Energy/deliv. (proxy)	Input bytes
AODV	0.7982	0.7357	3.857	15.420	143
OLSR	0.7657	0.7039	3.139	27.164	138
Greedy Geographic	0.6832	0.6371	3.568	25.761	2284
Evo-QGeo (Adapted)	0.8869	0.8154	4.557	2.248	6452
	(0.8850–0.8887)	(0.8132–0.8175)	(4.465–4.650)	(2.239–2.258)	
RDQN-HERP (Adapted)	0.7251	0.5996	6.163	3.348	4977
	(0.5977–0.8526)	(0.4527–0.7465)	(5.674–6.652)	(2.714–3.983)	
GAT-GRU-DDQN	0.6758	0.5817	5.494	13.191 (–13.066–	4451
	(0.5660–0.7855)	(0.4841–0.6793)	(5.116–5.872)	39.448)	
Fast-DiSwitch	0.8872	0.8150	4.486	2.369	6437
	(0.8682–0.9062)	(0.7910–0.8390)	(4.301–4.672)	(2.229–2.510)	
DiSwitch	0.9053	0.8376	4.264	2.228	4821
	(0.9039–0.9067)	(0.8363–0.8390)	(4.183–4.346)	(2.225–2.231)	(4691–4952)

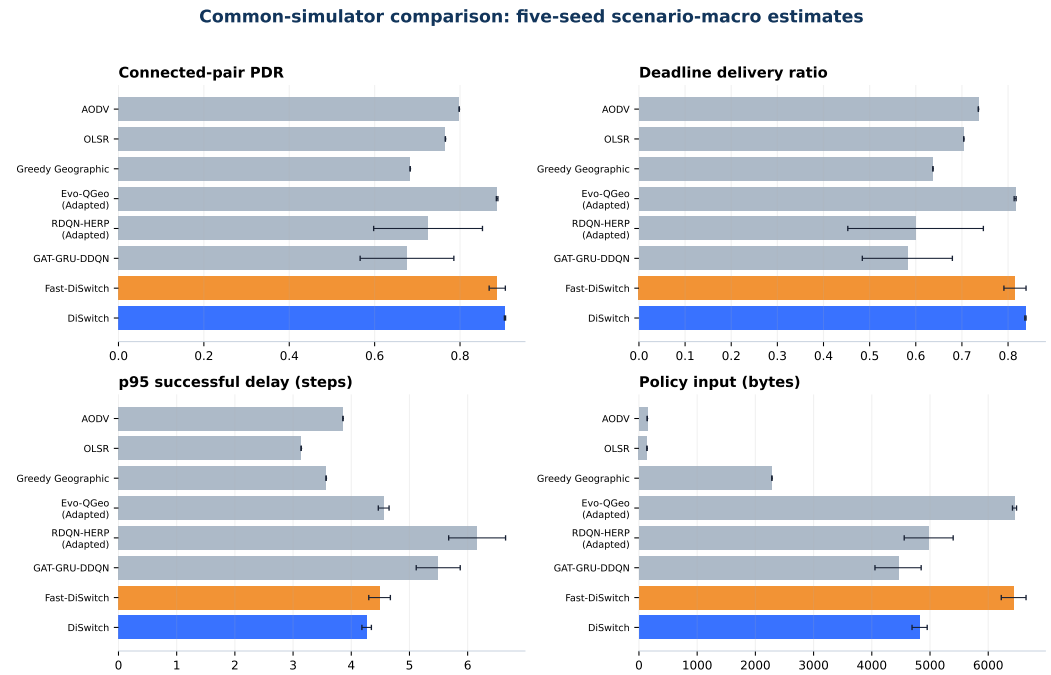


Figure 6. Common-simulator comparison. Error bars are five-seed 95% t intervals over scenario-macro summaries. Small input size for AODV/OLSR is not wireless routing overhead.

Against Evo-QGeo, the strongest adapted baseline by connected PDR, DiSwitch improves connected PDR by 0.01841 (95% CI 0.01520–0.02161) and deadline ratio by 0.02229 (0.01890–0.02567). DiSwitch also has 0.2929 fewer p95 successful-delay steps (0.1503–0.4354), a 0.02031 lower energy proxy per delivered packet (0.00826–0.03237), and 1,630 fewer policy-input bytes (1,507–1,753). These are paired scenario-macro contrasts; they do not imply that every scenario or every packet is superior.

6.2. Scenario Robustness and Failure Localization

The scenario heatmap in Figure 7 shows why aggregate reliability alone is insufficient. The predictive recovery mechanism is most useful in structural-hole and link-break settings where the normal geographic-residual policy can collapse. DiSwitch remains vulnerable in the predictive-break-plus-loss scenario, where Evo-QGeo retains an advantage. This failure persisted after switch recalibration and predictive-student retraining, showing a persistent failure in the tested interventions. These interventions do not establish whether the limiting cause is the policy family, training coverage, or simulator dynamics.

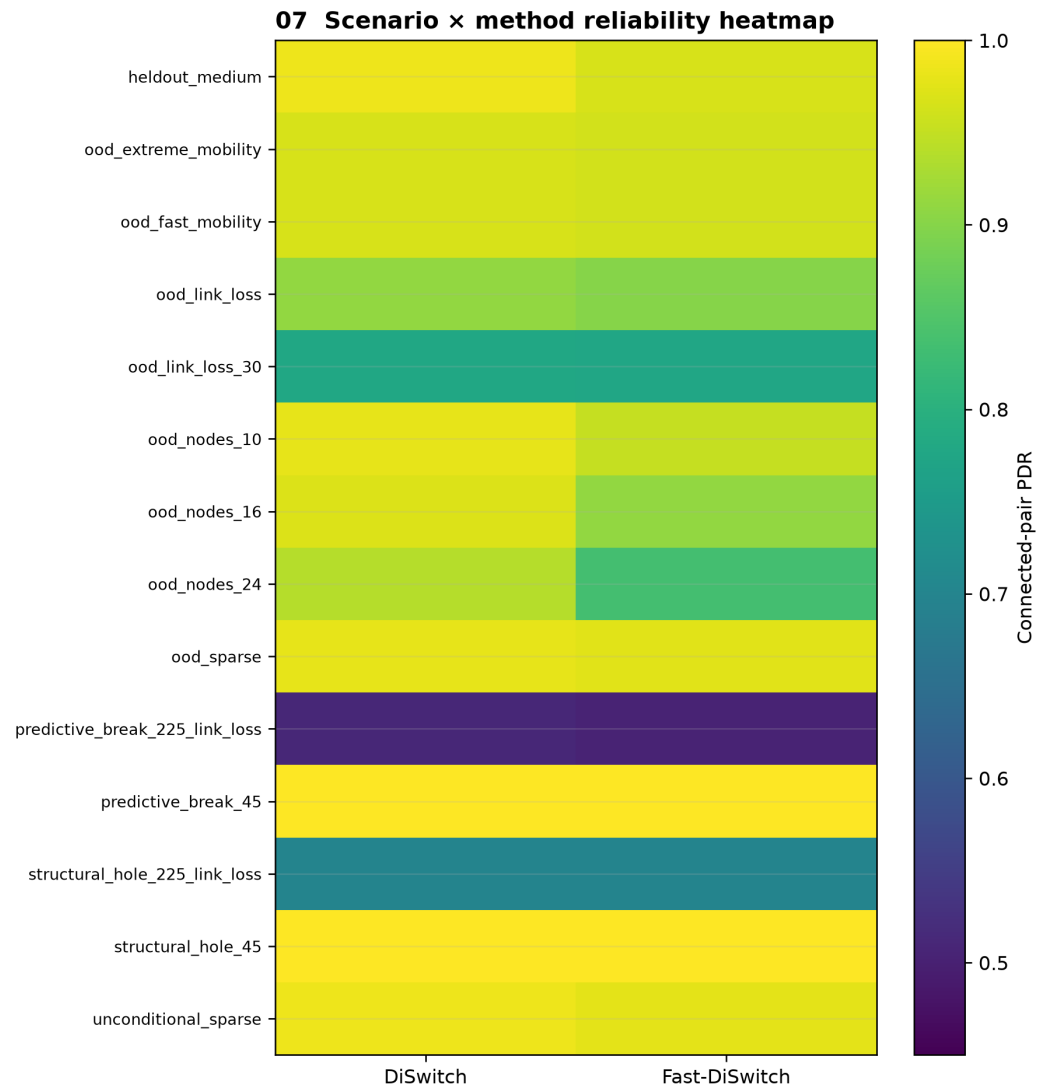


Figure 7. Five-seed scenario-level reliability. The heatmap localizes both predictive recovery and the node-scale or link-loss conditions in which approximations degrade.

The three node-count-shift cases provide a limited generalization test, not a scalability proof. The full graph teacher is absent online, but the local tensor size still depends on the maximum candidate capacity and neighborhood representation. Larger swarms with higher local degree, realistic radio interference, and multiple concurrent flows require separate validation.

6.3. Ablation Evidence

Figure 8 reports paired component contrasts. Comparisons between historical checkpoints can confound architecture, training data, and tuning budget. They do not isolate the causal contribution of distillation without a matched no-distillation control trained under the same budget. Removing the switch, relying on the predictive prior alone, or using the geographic-residual predecessor changes different scenario families. The result is consistent with the architectural interpretation: global-to-local distillation supplies a capable normal policy, predictive features offer recovery information, and the calibrated switch prevents always-on predictive behavior from replacing safe nominal decisions indiscriminately.

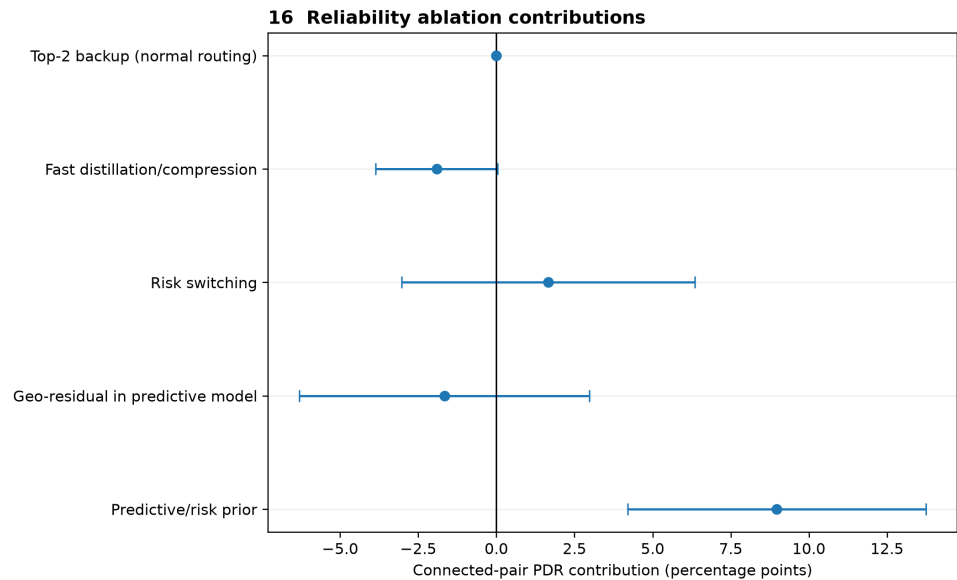


Figure 8. Directional component contributions with five-seed 95% intervals. Positive values favor the complete DiSwitch policy under the metric’s preferred direction.

Deletion/move candidate D04. The project history includes additional negative interventions: increasing the Fast-DiSwitch hidden dimension from 32 to 48 worsened the 24-node OOD case; density-augmented distillation improved node-count cases but introduced seed-specific predictive-break failures; switch recalibration under stochastic loss did not change the limiting outcome; and retraining the predictive student on that loss family reproduced the same failure pattern. These results are not promoted as separate contributions, but they constrain the explanation and motivate worst-scenario reweighting and action-space shielding.

6.4. Semantics-Preserving Reuse and Batch-One Decision Latency

Table 3 reports the same-session A100 results. The DiSwitch CUDA p95 is 5.836 ms, compared with 10.004 ms for legacy repeated execution: a 41.66% reduction. On the Colab host CPU, the corresponding values are 2.208 and 3.775 ms, a 41.51% reduction. The policy result is unchanged because the speedup removes duplicate evaluation rather than changing either branch.

Table 3. Same-session batch-one end-to-end decision latency. Values are means of five seed-specific statistics; paired p95 change is relative to DiSwitch and positive means faster.

Variant	Device	Mean (ms)	p50	p95	p99	p95 change
DiSwitch	CPU	2.168	2.161	2.208	2.344	reference
Early Exit	CPU	2.366	2.356	2.427	2.581	−9.90%
Fast-DiSwitch	CPU	0.812	0.814	0.847	0.927	+61.66%
Fast-DiSwitch + Top-2	CPU	0.853	0.852	0.881	0.933	+60.11%
Buffered DiSwitch	CPU	2.187	2.179	2.244	2.370	−1.62%
Legacy repeated	CPU	3.698	3.687	3.775	3.935	−70.97%
DiSwitch	CUDA	5.671	5.648	5.836	6.101	reference
Early Exit	CUDA	6.198	6.172	6.386	6.663	−9.42%
Fast-DiSwitch	CUDA	1.957	1.950	2.005	2.120	+65.65%
Fast-DiSwitch + Top-2	CUDA	2.040	2.032	2.092	2.183	+64.16%
Buffered DiSwitch	CUDA	5.626	5.602	5.788	6.020	+0.83%
Legacy repeated	CUDA	9.741	9.706	10.004	10.376	−71.43%

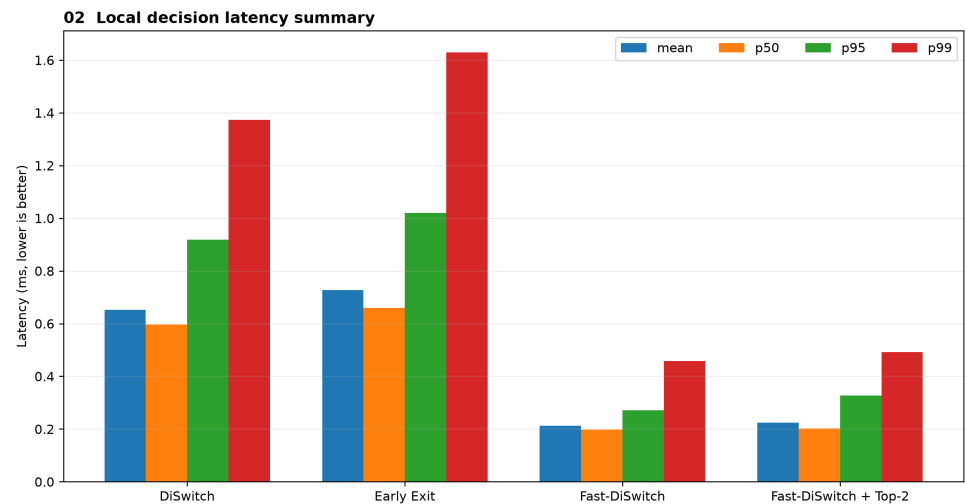


Figure 9. Latency summary across the verified implementations. The low Fast-DiSwitch latency must be read together with its reliability failure.

The paired CUDA p95 reduction for Buffered DiSwitch is only 0.83% (95% t CI 0.21–1.44); it is classified as “hold” because the effect is small and is negative on CPU. Early Exit is 9.42% slower on CUDA (CI –10.62 to –8.22). Figure 10 shows the uncertainty and direction of these effects.

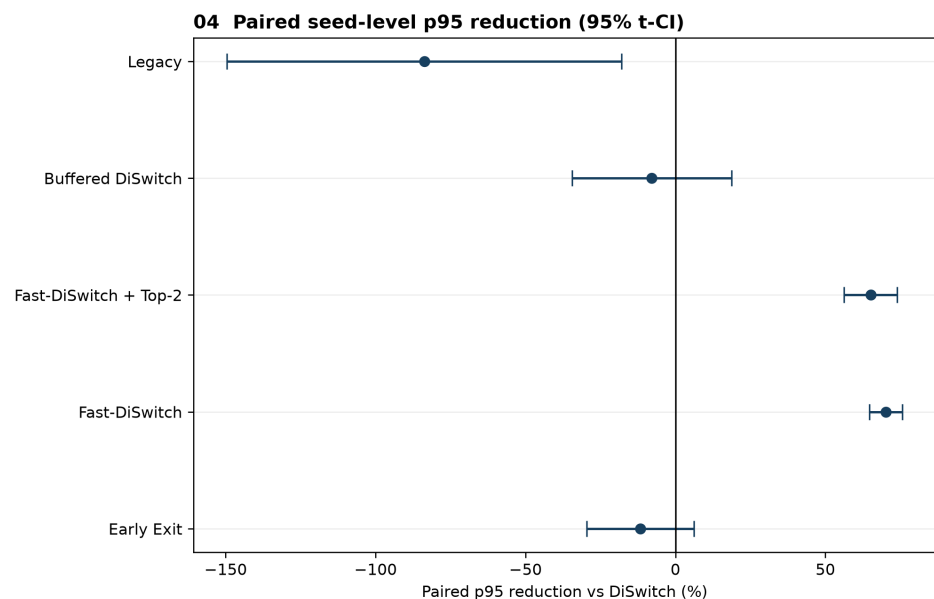


Figure 10. Paired p95 latency effects versus DiSwitch. Positive values favor the candidate. Intervals are based on five matched checkpoints, not on 10,000 pseudo-independent individual calls.

6.5. Why CUDA Is Slower Than the CPU

DiSwitch is 62.16% lower in p95 latency on the A100-session host CPU than on CUDA (2.208 versus 5.836 ms). This result is consistent with a fine-grained batch-one workload. The model has small matrix operations, while each CUDA decision launches many kernels, constructs or transfers small tensors, and materializes scalar values for Python-side conditions and action extraction. This pattern is consistent with launch and synchronization overhead offsetting the benefit of parallel arithmetic; the measurements do not isolate a causal contribution for each operator.

Operator profiling supports this attribution. Across 30 DiSwitch profiled calls, the local-scalar materialization operator occurs 1,050 times and is paired with 1,050 device-

to-host copies. Early Exit increases these counts to 1,140; Fast-DiSwitch reduces them to 300. The profiler cannot apportion the complete synchronized p95, but it identifies a plausible synchronization mechanism and explains why Python conditional execution does not satisfy $T_g < p_{\text{skip}} T_P$.

Figure 11 shows the device comparison. It does not imply that every CPU is faster than every embedded GPU. Deployment must be remeasured on the target flight computer, preferably including power and forwarding-table update time.

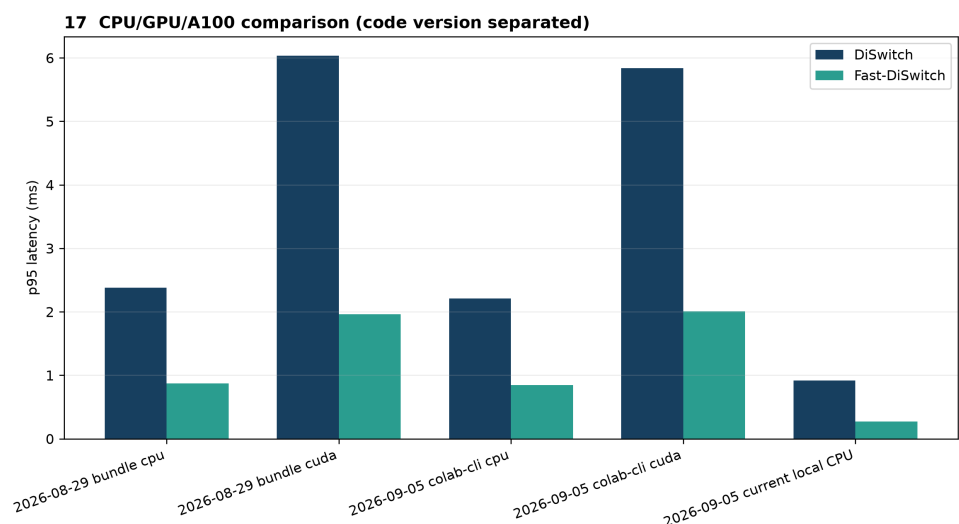


Figure 11. CPU and CUDA decision latency in the verified A100 session, with an older bundle retained only as an independent reproducibility reference.

6.6. Why Fast-DiSwitch Does Not Replace DiSwitch

Fast-DiSwitch reduces CUDA p95 to 2.005 ms, 65.65% below DiSwitch, but connected PDR decreases from 0.90529 to 0.88720. The directional paired change is -0.01809 with 95% CI $[-0.03606, -0.00011]$, so its lower endpoint fails the study-defined -0.005 non-inferiority criterion. Deadline ratio decreases by 0.02264 with CI $[-0.04625, 0.00096]$; successful p95 delay and energy proxy also worsen. The compact model is therefore a latency–quality research candidate, not the final method.

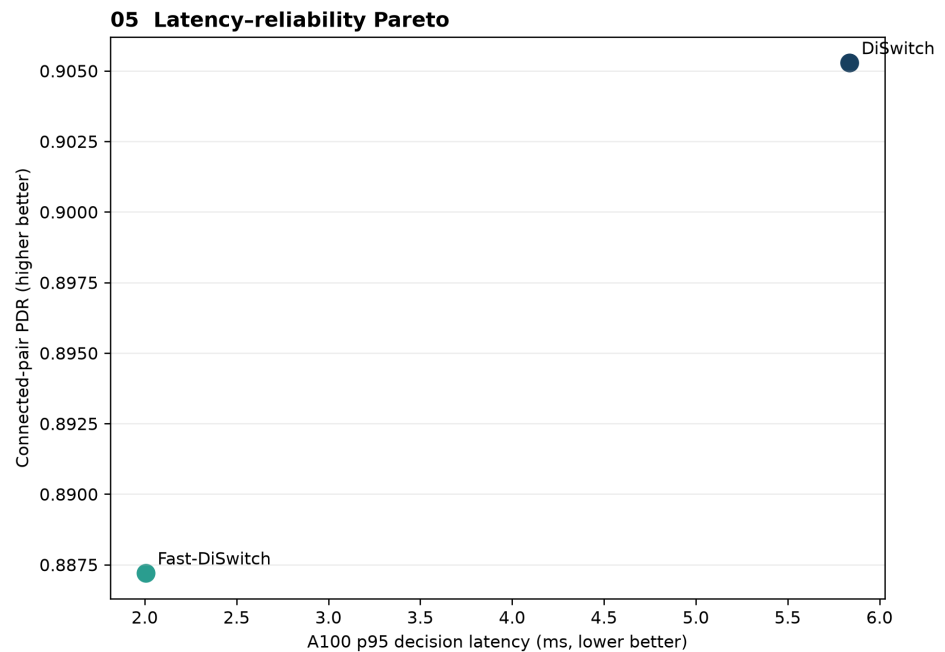


Figure 12. A100 p95 decision latency versus connected-pair PDR. The upper-left region is preferred; DiSwitch and Fast-DiSwitch represent distinct quality–latency choices.

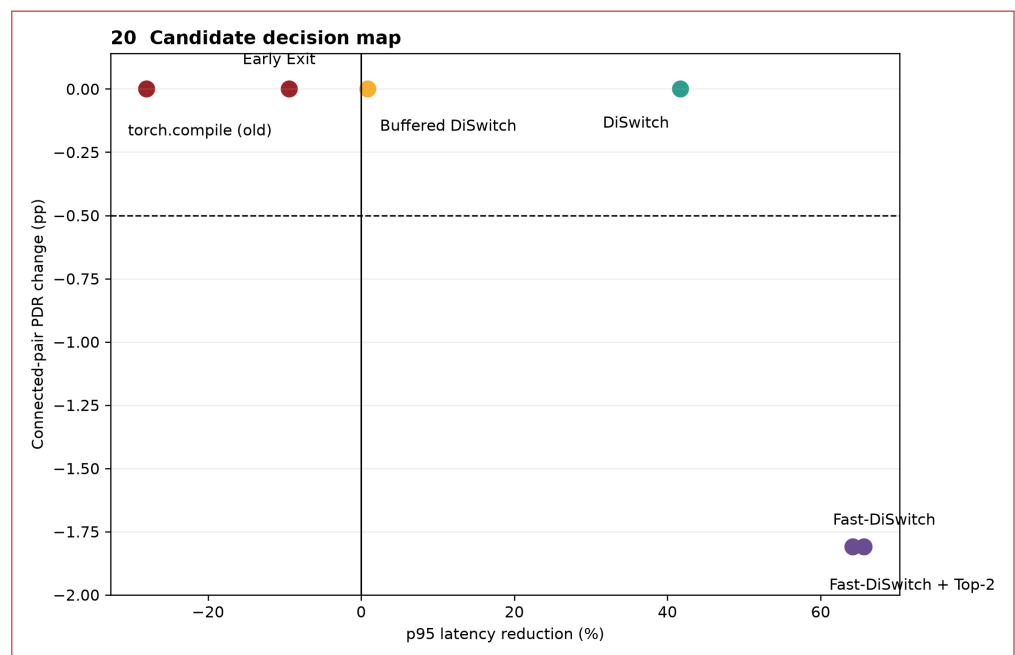


Figure 13. Deletion/move candidate D03. Redundant with the reliability–latency plot and acceptance discussion. Decision record for optimization candidates. DiSwitch is accepted because it preserves the validated policy behavior; Fast-DiSwitch fails reliability, and Python Early Exit fails latency.

6.7. Early-Exit Calibration as a Negative Result

At a zero danger margin, Early Exit skips the predictive branch for 32,285 of 43,467 decisions (74.27%) and produces no episode-trajectory mismatch over 14,000 paired episodes for the current checkpoints and scenarios. Positive margins increase the skip rate but cause action divergence. Figure 14 shows this empirical calibration boundary.

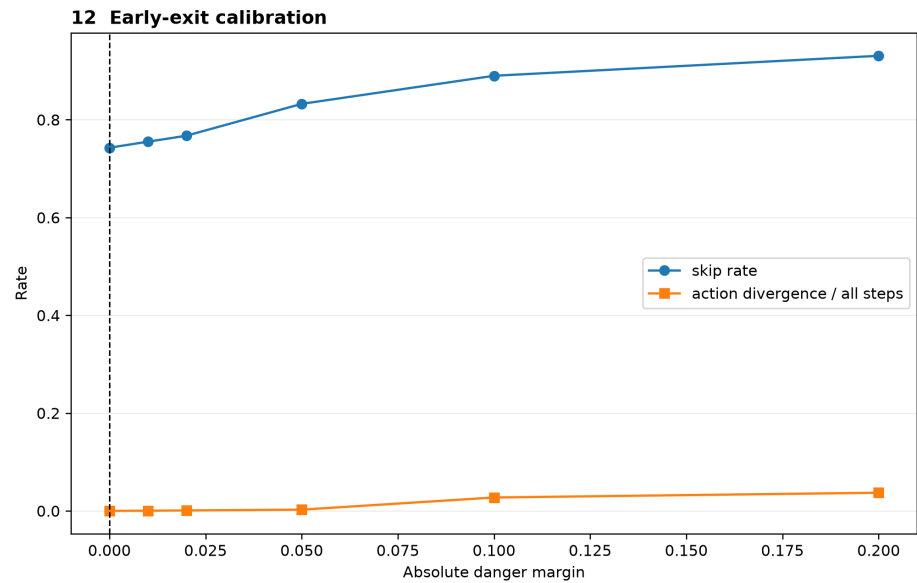


Figure 14. Early-exit calibration. Zero observed divergence at margin zero is empirical evidence for the evaluated trajectories, not formal equivalence for unseen states.

Despite the high skip rate, Early Exit is slower. Its Python branch requires scalar materialization and control flow before saving a small predictive multilayer perceptron call. This negative result rules out the naive “high skip rate implies low latency” argument and motivates a tensor-only conditional graph, such as a deployment-runtime conditional operator, for future work.

7. Discussion

7.1. Answering the Research Questions

First, the common-simulator evidence supports the strict-local architecture: DiSwitch provides the strongest scenario-macro reliability and deadline delivery among the eight implementations, with paired advantages over the strongest adapted baseline. The claim is deliberately limited to the evaluated simulator, information contracts, and scenarios.

Second, recovery is not attributable to a single black-box network. The geographic prior stabilizes nominal forwarding, the distilled residual transfers global structural preference to local features, predictive risk exposes impending breaks, and the calibrated switch determines when the predictive prior should replace the nominal choice. The ablation and scenario heatmap show complementary roles and a remaining predictive-break-with-loss weakness.

Third, semantics-preserving execution reuse materially reduces the avoidable portion of decision latency, but the remaining model is too fine-grained to benefit automatically from the A100 at batch one. Host-device synchronization and launch overhead are part of the routing system, not incidental benchmarking noise. For online packet routing, batching decisions can change the control semantics or add waiting delay; batch one is therefore the deployment-relevant primary measurement.

Fourth, neither Fast-DiSwitch nor the tested Early Exit is an acceptable replacement. Fast-DiSwitch wins the timer and loses the study-defined quality gate. Early Exit preserves observed trajectories at its conservative threshold but loses the timer. DiSwitch is the final proposed implementation because it is the only tested change that offers a large latency reduction relative to the duplicated legacy path without changing the policy.

7.2. End-to-End Delay Versus Decision Latency

Both quantities should be measured, but they answer different questions. Network end-to-end delay can be written schematically as

$$T_{\text{E2E}} = \sum_{h=1}^H (T_{\text{queue},h} + T_{\text{MAC},h} + T_{\text{tx},h} + T_{\text{prop},h} + T_{\text{decision},h} + T_{\text{update},h}). \quad (14)$$

The simulator's successful delay in steps captures the route-level effect of hop count, retries, and connectivity under its abstraction. The microbenchmark isolates observation-to-action software latency. A future hardware-in-the-loop evaluation should add routing-table update and real radio timing. Combining these terms into one unlabeled "delay" would make comparison with RoutePPO, RLFR, Predictive-Q, and simulation papers misleading.

7.3. Practical Deployment Guidance

The present evidence supports a CPU-first batch-one deployment profile for the current PyTorch implementation. The unified decision API should be the only adapter entry point, and regression tests should assert one normal and one predictive forward per decision. A GPU path should avoid repeated small tensor creation and should keep gate and action extraction tensor-resident. If multiple independent flows can be accumulated without violating their deadlines, micro-batching may amortize launches, but the queuing time introduced by batching must be included in end-to-end latency.

The local observation is compatible with decentralized forwarding, yet feature acquisition is not free. Link-lifetime and onward-connectivity estimates require beacons or local prediction. The current policy-input byte metric measures memory presented to the model, not the over-the-air bytes required to maintain those estimates. A complete implementation must instrument both quantities.

7.4. Threats to Validity and Limitations

Simulation validity. The routing results come from a project simulator rather than flight or wireless testbed measurements. Radio, mobility, interference, clocking, and energy abstractions may not reproduce real vehicles. The adapted baselines are controlled common-environment implementations, not authoritative reproductions of every original paper.

Statistical power. Five training seeds permit paired effect estimation but cannot produce a two-sided exact sign-flip $p < 0.05$. Some deterministic baselines have zero between-seed variance because their policy does not depend on training seed. Future confirmation should use at least 10 independent seeds, randomized runtime block order, and hierarchical bootstrap over seeds and scenario families.

Aggregation. Scenario-macro weighting gives each scenario equal influence. A traffic-weighted operational deployment could produce different priorities. Failure-penalized energy is particularly sensitive to aggregation and must not be compared with raw transmission-only energy without restating the formula.

Latency trace granularity. The primary A100 archive stores seed/component aggregates rather than every one of the 2,000 raw timings. It includes warm-up and synchronized percentiles, but a stricter replication should retain the randomized raw timing order. The short operator profile is attribution evidence, not a latency sample.

Hardware representativeness. An A100 is not an onboard computer. Its value here is controlled profiling of CUDA behavior, not a claim of flight readiness. Jetson Orin,

Raspberry Pi-class processors, or the target autopilot companion computer should be measured under thermal and power constraints.

Equivalence scope. DiSwitch is equivalent to the legacy two-branch decision under deterministic evaluation assumptions. Early Exit's zero mismatch is only empirical for the current checkpoints and trajectory set. Fast-DiSwitch is explicitly approximate. No result establishes formal safety for all reachable network states.

Provenance. The synthesis manifest records the source commit and a dirty-file list. The archive is internally complete and cross-experiment row counts were verified, but the non-clean generation state is a reproducibility weakness. A release candidate should rerun or freeze the final bundle under a clean tag and publish hashes.

8. Future Work

The immediate systems direction is tensor-only conditional computation. The normal branch, danger computation, conditional predictive branch, and action selection should be captured in one compiled graph without Python `.item()` calls. Its acceptance criterion remains identical to Equation (3). A second direction is conservative action-space pruning: remove only disconnected or provably risk-infeasible candidates before the policy head, then test whether the action remains identical on exhaustive replay.

Approximate acceleration requires a better training objective. Candidate losses include branch-logit distillation, switch-logit calibration, pairwise candidate ranking, worst-scenario reweighting, and selective cascades that invoke Fast-DiSwitch only at high confidence and low risk. The cascade must be calibrated on held-out scenario families and confirmed on new seeds, not tuned on the final evaluation archive.

The next empirical stage should combine a clean 10-seed simulator run with target-device measurements. This requires separately recording model inference, observation construction, routing-table update, radio-control traffic, end-to-end packet delay, energy in joules, temperature, and power mode. RoutePPO and RLFR suggest useful forwarding-plane and small-device precedents [14,15]; larger-scale and multipath work suggests expanding density, concurrency, and failover conditions [18,19].

9. Conclusions

DiSwitch combines global-to-local policy distillation with risk-switched predictive recovery for UAV next-hop selection. In the evaluated common simulator, it achieved 0.9053 connected-pair PDR and 0.8376 deadline delivery and had positive paired differences relative to adapted Evo-QGeo. The batch-one benchmark gave mean seed-specific p95 decision latencies of 5.836 ms on A100 CUDA and 2.208 ms on the session's CPU. Fast-DiSwitch reduced CUDA latency to 2.005 ms but failed the study-defined reliability criterion. These findings are restricted to the archived checkpoints, simulator scenarios, and measured session. Establishing operational utility requires a reproducible release, controlled ablations, and target-device and wireless-network evaluation.

Author Contributions: Kyung Eun Kim and Howon Lee must confirm the final CRediT assignments and approval of the manuscript.

Funding: Provide the funding agency and grant number, or confirm that no external funding was received.

Institutional Review Board Statement: Not applicable to the reported simulation study.

Informed Consent Statement: Not applicable.

Data Availability Statement: The publication package includes figure-source data and analysis summaries. Provide the archival DOI and repository URL for code, configurations, checkpoints, and raw results, or specify approved access conditions.

Acknowledgments: OpenAI Codex assisted with English drafting, LaTeX restructuring, and figure-layout preparation in the abstract, methods, related work, results, and supplementary material [20]. The authors are responsible for verifying the final content.

Conflicts of Interest: Both authors must verify the final conflict-of-interest declaration.

References

1. Alam, M.M.; Moh, S. Joint Trajectory Control, Frequency Allocation, and Routing for UAV Swarm Networks: A Multi-Agent Deep Reinforcement Learning Approach. *IEEE Transactions on Mobile Computing* **2024**, *23*. <https://doi.org/10.1109/TMC.2024.3403890>.
2. Wu, H.; Wu, S.; Li, A.; Meng, S.; Zhang, Q. AoI-Aware Joint Sampling-Buffering-Routing Optimization for Autonomous UAV Swarms via a MARL Approach. *IEEE Transactions on Mobile Computing* **2026**. <https://doi.org/10.1109/TMC.2026.3699777>.
3. Jia, Z.; He, S.; Zhu, Q.; Wang, W.; Wu, Q.; Han, Z. Trusted Routing for Blockchain-Empowered UAV Networks via Multi-Agent Deep Reinforcement Learning. *IEEE Transactions on Communications* **2025**. <https://doi.org/10.1109/TCOMM.2025.3597655>.
4. Cui, Y.; Zhang, Q.; Feng, Z.; Wei, Z.; Shi, C.; Yang, H. Topology-Aware Resilient Routing Protocol for FANETs: An Adaptive Q-Learning Approach. *IEEE Internet of Things Journal* **2022**, *9*, 18632–18650. <https://doi.org/10.1109/JIOT.2022.3162849>.
5. Zhang, J.; Huang, H.; Shi, C.; Zhang, Y.; Fan, B.; Li, D. Spatiotemporal-Aware Resilient Routing Algorithm Based on Graph Attention Double DQN. In Proceedings of the Proceedings of the 17th International Conference on Wireless Communications and Signal Processing, 2025. <https://doi.org/10.1109/WCSP68525.2025.1010249>.
6. Shou, Y.; Liu, D.; Hou, X. Adaptive Intelligent Routing in Ultra-High-Speed FANETs Using Deep Reinforcement Learning. *IEEE Transactions on Vehicular Technology* **2026**. <https://doi.org/10.1109/TVT.2026.3668740>.
7. Tho, M.C.; Ly, N.T.H.; Binh, L.H.; Vo, T.T. QLR-FANET: A Q-learning and Rate Control-Based Routing Protocol for Flying Ad Hoc Network. *ETRI Journal* **2025**, *47*, 1015–1027. <https://doi.org/10.4218/etrij.2024-0298>.
8. Sharvari, N.P.; Das, D.; Bapat, J.; Das, D. Improved Q-Learning-Based Multi-Hop Routing for UAV-Assisted Communication. *IEEE Transactions on Network and Service Management* **2025**, *22*. <https://doi.org/10.1109/TNSM.2024.3522153>.
9. Dong, Z.; Sun, H.; Tao, Y.; Zhai, D. Predictive-Q Learning Based Interference-and-Mobility Aware Dual-Path Routing for UAV Swarm Networks with Mobile Edge Computing. *Computers, Materials & Continua* **2026**, *88*. <https://doi.org/10.32604/cmc.2026.084301>.
10. Ke, Y.; Huang, K.; Qiu, X.; Song, B.; Xu, L.; Yin, J.; Yang, Y. Distributed Routing Optimization Algorithm for FANET Based on Multiagent Reinforcement Learning. *IEEE Sensors Journal* **2024**, *24*. <https://doi.org/10.1109/JSEN.2024.3415127>.
11. Wang, C.; Liu, Y.; Song, R.; Yu, X.; Wang, X.; Chen, X.; Liu, J. Joint Optimization of Routing and Power Control in UAV Networks Via Graph Reinforcement Learning. In Proceedings of the Proceedings of the 17th International Conference on Wireless Communications and Signal Processing, 2025. <https://doi.org/10.1109/WCSP68525.2025.1010551>.
12. Zheng, F.; Wei, J.; Li, S.; Yu, P.; Guo, S.; Zhao, J.; Huang, Y. A Dual-Layer Deep Reinforcement Learning Routing Approach for Integrated UAV and Satellite IoT Networks. *IEEE Internet of Things Journal* **2026**, *13*. <https://doi.org/10.1109/JIOT.2025.3602147>.
13. Sun, S.; Cao, C.; Lyu, Y.; Xu, X.; Huang, R.; Xu, Y.; Tang, S.; Fu, C.; Wu, W. Intelligent Maintenance and Routing Decision Making for UAV Clusters in Harsh Environments. *Drones* **2026**, *10*, 455. <https://doi.org/10.3390/drones10060455>.
14. Li, J.; Xiao, L.; Qi, X.; Lv, Z.; Chen, Q.; Liu, Y.J. Reinforcement Learning Based Energy-Efficient Fast Routing for FANETs. *IEEE Transactions on Communications* **2024**, *72*. <https://doi.org/10.1109/TCOMM.2024.3409561>.
15. Cürmen, N.; Okay, F.Y.; Özdemir, S. RoutePPO: eBPF-Based Proximal Policy Optimization for Adaptive Routing in UAV Swarm Networks. In Proceedings of the Proceedings of the International Conference on Smart Applications, Communications and Networking, 2026. <https://doi.org/10.1109/SMARTNETS69662.2026.11604730>.

16. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal Policy Optimization Algorithms, 2017, [1707.06347]. <https://doi.org/10.48550/arXiv.1707.06347>. 558 559
17. Hinton, G.; Vinyals, O.; Dean, J. Distilling the Knowledge in a Neural Network. In Proceedings of the NIPS Deep Learning and Representation Learning Workshop, 2015, [1503.02531]. 560 561
18. Zhang, X.; Lei, L.; Shen, G.; Fan, J.; Pan, Y.; Cao, P. A Large-Scale UAV Swarms Routing Approach With Deep Reinforcement Learning Leveraging Intelligent Endogenous Network. *IEEE Transactions on Vehicular Technology* **2026**. <https://doi.org/10.1109/TVT.2026.3672174>. 562 563 564
19. Zhao, Z.; Zhang, T.; Xu, X.; Li, J.; Liu, Y.; Xing, W. Multipath Routing for Multi-Hop UAV Networks, 2026, [2601.10299]. <https://doi.org/10.48550/arXiv.2601.10299>. 565 566
20. OpenAI. Codex. Software system, 2026. Accessed: September 5, 2026. 567